

Module 1: Frontend PWA

What is a Progressive Web App (PWA)?

A web application with native app-like features

Key components:

- ▶ Service Worker: Background script for caching & offline
- ▶ Web App Manifest: JSON file defining app metadata
- ▶ HTTPS: Required for security features

Benefits:

- ▶ Installable on home screen
- ▶ Works offline or with poor connectivity
- ▶ Push notifications (optional)
- ▶ No app store approval needed

Browser APIs: Camera & Geolocation

MediaDevices API (Camera)

- ▶ `navigator.mediaDevices.getUserMedia()`
- ▶ Returns a Promise with `MediaStream`
- ▶ Requires user permission (once per origin)

Geolocation API:

- ▶ `navigator.geolocation.getCurrentPosition()`
- ▶ Returns latitude, longitude, accuracy
- ▶ Requires explicit user consent

Both APIs

- ▶ Only work on HTTPS (or localhost)
- ▶ User can revoke permission anytime

WebRTC Fundamentals

`getUserMedia()` returns a `MediaStream` object

- ▶ `MediaStream` contains tracks (video, audio)

To capture a frame:

- ▶ Attach stream to `<video>` element
- ▶ Draw video frame to `<canvas>`
- ▶ Export canvas as base64 image

Constraints object controls resolution:

- ▶ `{ video: { width: 640, height: 480 } }`
- ▶ `facingMode: 'user'` (front) or `'environment'` (back)

Always release stream when done:

- ▶ `track.stop()`

State Management & Token Security

Where to store JWT tokens?

- ▶ localStorage: Persists, XSS vulnerable
- ▶ sessionStorage: Tab-scoped, XSS vulnerable
- ▶ Memory (variable): Safe, lost on refresh
- ▶ HttpOnly Cookie: XSS-safe, CSRF concerns

For this project:

- ▶ Access token: Memory or sessionStorage
- ▶ Refresh token: HttpOnly cookie (ideal) or secure storage

NEVER store tokens in URL or logs

Clear tokens on logout

Offline-First Design Patterns

Service Worker caching strategies

- ▶ Cache First: Serve from cache, fall back to network
- ▶ Network First: Try network, fall back to cache
- ▶ Stale While Revalidate: Serve cache, update in background

What to cache:

- ▶ Static assets (JS, CSS, images)
- ▶ App shell (HTML structure)

What NOT to cache:

- ▶ Authentication responses
- ▶ Check-in submissions (must be real-time)

Use IndexedDB for structured offline data

User Consent & Permission Handling

Permission states:

- ▶ 'granted', 'denied', 'prompt'

Check permission before requesting:

- ▶ `navigator.permissions.query({ name: 'camera' })`

Best practices:

- ▶ Explain WHY before requesting permission
- ▶ Don't request all permissions at once
- ▶ Handle denial gracefully with fallback UI
- ▶ Never trick users into granting permissions

Record consent in backend (camera_consent, geolocation_consent)
